

Spring Boot Hour 4 Configuration & Profiles

Spring Boot — 8-Hour Plan · HOUR 4

Configuration & Profiles: One JAR, Many Environments

Goal: your app runs on your laptop with a local MySQL, on a test server with a test DB, and in production with the real DB — same code, three different settings. This hour is about how Spring Boot makes that painless, and it's full of small questions interviewers use as warm-ups (get them instantly right = strong first impression).

Carry-over nugget from Hour 3's quiz: calling one `@Bean` method from another does NOT create a duplicate object. `@Configuration` classes are secretly proxied by Spring, so the call is intercepted and returns the container's one managed singleton. 🐼
"`@Configuration` classes are proxied so `@Bean` method calls between beans return the managed singleton instead of creating duplicates."

4.1 Why config lives OUTSIDE the code

Imagine the database password hardcoded in Java:

```
String dbUrl = "jdbc:mysql://localhost:3306/demo"; // 🐼 hardcoded
```

Three disasters waiting:

1. **Every environment change = recompile.** Test server uses a different DB → you edit code, rebuild, redeploy. For a *setting*.
2. **Secrets in the repo.** The password is now in Git history. Forever. Anyone with repo access has prod credentials.
3. **One build can't travel.** The JAR built for dev physically cannot run in prod.

The principle: **code is WHAT the app does; configuration is WHERE/HOW it runs. Keep them separate.** Then one identical JAR travels dev → test → prod, and only the settings around it change.

Analogy: a gas stove. The stove (code) is manufactured once, identically for everyone. The knobs (configuration) — flame size, which burner — are set by each kitchen. Nobody re-manufactures the stove to lower the flame.

Spring Boot's knobs live in one file you already know: `src/main/resources/application.properties`.

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/demo
spring.datasource.username=root
spring.datasource.password=root123
spring.jpa.hibernate.ddl-auto=update
logging.level.com.example.demo=DEBUG
```

Connect this to Hour 1: **this file is how you override auto-configuration's defaults.** Boot auto-configures the port to 8080; you disagree → one line, `server.port=9090`. The camera's Auto mode, with manual dials available.

4.2 properties vs yaml — same knobs, two notations

The exact same settings can be written as `application.yml`:

```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/demo
    username: root
    password: root123
```

They are 100% equivalent in power. `.properties` = flat `a.b.c=value` lines. `.yaml` = the same keys as an indented tree — repeated prefixes (`spring.datasource...` three times) get written once. YAML is easier to read for deep nesting; its only danger is that **indentation is meaning** — one wrong space = broken config.

🗨 INTERVIEW CHECKPOINT

Q: "application.properties vs application.yml?" 🐼: "Functionally identical — two syntaxes for the same property sources. YAML is hierarchical and less repetitive for nested keys; properties is flat and more typo-tolerant. Team preference decides; if both exist, properties wins over yml."

4.3 Reading config in code — @Value vs @ConfigurationProperties

Setting values is half the story; your beans need to **read** them.

Way 1 — @Value: grab one value

```
@Service
public class MailService {
    @Value("${app.mail.from}")           // ${...} = "look this key up in the environment"
    private String fromAddress;

    @Value("${app.mail.retries:3}")     // ":3" = default if the key is missing (nice touch to mention!)
    private int retries;
}
```

Perfect for one or two values. But imagine 15 related settings — 15 scattered @Value fields across classes, each a stringly-typed key that typos silently at runtime. Messy.

Way 2 — @ConfigurationProperties: bind a whole GROUP to one class

```
app.mail.from=noreply@demo.com
app.mail.retries=5
app.mail.subject-prefix=[DEMO]
```

```
@Component
@ConfigurationProperties(prefix = "app.mail") // "bind every key under app.mail into me"
public class MailProperties {
    private String from;           // ← app.mail.from lands here (matched by name)
    private int retries;          // ← app.mail.retries
    private String subjectPrefix; // ← app.mail.subject-prefix (kebab-case auto-maps to camelCase!)
    // getters & setters
}
```

Now inject MailProperties anywhere — one typed object, IDE autocomplete, validation possible (@Validated + @Min on fields), all mail config in one place.

Analogy: @Value = asking the storekeeper for items one by one ("one onion... now one tomato..."). @ConfigurationProperties = handing him the list titled "vegetables" and receiving the whole labeled basket.

🗨 INTERVIEW CHECKPOINT — very frequently asked

Q: "@Value vs @ConfigurationProperties?" 🐼: "@Value injects individual keys — fine for one-offs. @ConfigurationProperties binds a whole prefix into a typed class — grouped, type-safe, supports validation and relaxed binding (kebab-case to camelCase). For any real group of settings, @ConfigurationProperties is the recommended approach."

4.4 The loading ORDER — who wins when the same key is set twice?

Here's the scenario that makes this section matter. Your application.properties says server.port=8080. Ops runs:

```
java -jar app.jar --server.port=9999
```

Which port wins? **9999**. Spring Boot reads MANY property sources and stacks them by priority. Simplified (memorize this order, high beats low):

- | | | |
|-------------------------------------|-------------------------|-------------|
| 1. Command-line arguments | --server.port=9999 | ← strongest |
| 2. OS environment variables | SERVER_PORT=9999 | |
| 3. application-{profile}.properties | (profile-specific file) | |
| 4. application.properties | (the default file) | |
| 5. Coded defaults | | ← weakest |

The logic behind the order — this is the part to say in an interview: **the closer to deployment time, the higher the priority**. The properties file was written weeks ago at build time; the environment variable was set on THIS machine; the command-line flag was typed for THIS run. The most specific, most recent intent wins — without touching the JAR.

Analogy: company dress code. The employee handbook (application.properties) says "formals." Your department's memo (profile file) says "smart casual on Fridays." Today your manager says "client visit — wear a suit" (command line). You wear the suit. The instruction closest to the moment overrides the general rulebook — and the handbook doesn't get reprinted for one day.

💬 INTERVIEW CHECKPOINT

Q: "Same property in application.properties and as a command-line arg — which applies?" 🎯: "Command line — it's the highest-priority property source. General order: command line > environment variables > profile-specific file > application.properties > defaults. This lets ops override any setting per-deployment without rebuilding."

Q: "Where would you keep the production DB password?" 🎯: "Never in the repo — in environment variables or a secrets manager, injected at deploy time. Since env vars outrank the properties file, they cleanly override any placeholder." (Your handbook's best-practices box says exactly this — "never commit real credentials.")

4.5 Profiles — named bundles of settings per environment

The loading order handles one-off overrides. But dev vs prod differ in **many** settings at once — DB, port, logging, feature flags. Overriding 10 keys on the command line every time is silly. Enter **profiles**: a named settings-bundle you switch with ONE flag.

Step 1 — one file per environment, named `application-{profile}.properties` :

<code>application.properties</code>	← shared/common settings (always loaded)
<code>application-dev.properties</code>	← <code>server.port=8081</code> , local DB, logging DEBUG
<code>application-prod.properties</code>	← <code>server.port=80</code> , prod DB, logging WARN

Step 2 — activate one:

```
java -jar app.jar --spring.profiles.active=prod      # command line
# or: SPRING_PROFILES_ACTIVE=prod                  # environment variable
# or in application.properties: spring.profiles.active=dev (typical local default)
```

How it loads: `application.properties` first (the base), then `application-prod.properties` **on top** — profile file wins on conflicts (it's higher in the order from 4.4, see!). So: common stuff in the base file, only the *differences* in profile files.

Step 3 — profiles can switch BEANS too, not just values:

```
@Service
@Profile("dev")                // this bean EXISTS only when dev is active
public class FakePaymentService implements PaymentService { } // pretend-pay locally

@Service
@Profile("prod")
public class RazorpayPaymentService implements PaymentService { } // real money in prod
```

One profile active → only one of these beans exists → your Hour-3 "two beans, one interface" conflict never happens; the environment picks the implementation. (`@Profile({"dev", "test"})` = active in either.)

Analogy: stage lighting presets. Instead of adjusting 40 individual lights between scenes (overriding keys one by one), the operator presses "Preset: Night Scene" — one named button reconfigures everything. `spring.profiles.active` is the preset button.

💬 INTERVIEW CHECKPOINT

Q: "What are profiles and how do you use them?" 🎯: "Named configuration sets per environment. Common config in `application.properties`, environment differences in `application-{profile}.properties`; activate with `spring.profiles.active`. `@Profile` can also include or exclude whole beans per environment — e.g., a mock payment service in dev, the real one in prod."

Q: "How would you run the SAME jar in dev and prod?" 🎯: "That's the point of externalized config: one artifact, and at launch I pass `--spring.profiles.active=prod` plus secrets via environment variables. Nothing is rebuilt."

4.6 The 30-second story for this hour

"Configuration is externalized into `application.properties` or `yaml` so one JAR runs everywhere. I read values with `@Value` for one-offs and `@ConfigurationProperties` for typed groups. Property sources stack by priority — command line beats env vars beats profile files beats the base file — so ops can override anything at deploy time. Environments are handled with profiles: shared config in the base file, differences in `application-{profile}.properties` files, activated by `spring.profiles.active`, with `@Profile` even swapping bean implementations per environment. Secrets never enter the repo — they come from env variables or a secrets manager."

🔧 HOUR 4 DRILL — out loud

1. Why externalize configuration? (one JAR everywhere; no rebuilds for settings; secrets out of Git)
2. `properties` vs `yaml`? (identical power; flat vs tree; `properties` wins if both exist)
3. `@Value` vs `@ConfigurationProperties`? (one key vs typed group binding; prefer CP for groups)
4. What does `${app.retries:3}` mean? (look up key, default 3 if missing)
5. Priority order of property sources? (command line > env > profile file > base file > defaults — "closest to deploy time wins")
6. Where do prod passwords live? (env vars / secrets manager — never the repo)
7. How do profiles work, file-naming and activation? (`application-{profile}.properties`; `spring.profiles.active`)
8. What does `@Profile("dev")` on a bean do? (bean exists only when that profile is active — env-specific implementations)
9. Base file says X, profile file says Y — which applies? (profile file — loaded on top)
10. Bonus: why does calling one `@Bean` method from another not create a duplicate? (`@Configuration` is proxied — returns the managed singleton)

Next → Hour 5: Talking to the Database — JPA vs Hibernate vs Spring Data JPA untangled, entities, and how `findByIdName` writes SQL for you.